FORMATION DÉVELOPPEMENT WEB

Jour 1

Interfaces web

HTML, CSS, Bootstrap et JavaScript

Journée de formation · 5 heures

Objectifs de la formation

Développement, sécurisation et optimisation des applications web

01 **Développer une interface web**

HTML, CSS, Bootstrap et JavaScript.

02 **Créer et sécuriser une API Laravel**

Données, authentification et fichiers.

03 **Mesurer et améliorer les performances**

Cache avec Valkey et tests avec k6.

Formation de 5 jours, à raison de 5 heures par jour.

Projet fil rouge

Une application simple de gestion de documents

Aujourd’hui

Une interface responsive et des interactions dans le navigateur.

Les jours suivants

Une API Laravel, des utilisateurs, des fichiers et des tests.

Les documents ajoutés aujourd’hui sont temporaires.

Environnement de travail

Avant de commencer, vérifier les fichiers et les outils

01 **Un navigateur récent**

Chrome, Edge ou Firefox avec les outils de développement.

02 **Un éditeur de code**

Par exemple Visual Studio Code.

03 **Le dossier du projet**

Ouvrir apps/app-html-css-js dans l’éditeur.

Prérequis : notions de base en HTML, CSS et JavaScript.

Déroulement du jour 1

Répartition indicative pour 5 heures pédagogiques

20 min	Présentation et environnement
55 min	HTML, CSS, Flexbox et Grid
1 h 20	Interface Bootstrap
1 h 10	JavaScript et formulaire
40 min	Ressources locales et localStorage
35 min	Synthèse, correction et questions

Les pauses sont à prévoir séparément.

Objectifs de l’exercice

Une page de documents en HTML et CSS

01 **Structurer une page avec HTML5**

02 **Relier une feuille CSS et mettre en forme**

03 **Aligner la navigation avec Flexbox**

04 **Organiser des cartes avec CSS Grid**

Durée indicative : 55 minutes

HTML et CSS

Deux rôles complémentaires

HTML

Décrit la structure et le contenu de la page.

CSS

Définit la présentation et la disposition.

```
.carte {
  background-color: white;
  padding: 16px;
}
```

.carte cible les éléments ayant class="carte".

Les éléments HTML de la page

Chaque balise décrit le rôle d'un contenu

```
<header>
  <nav>...</nav>
</header>
<main>
  <section>...</section>
</main>
```

header

Contient l'en-tête de la page.

nav

Regroupe les liens de navigation.

main et section

Organisent le contenu principal et ses parties.

Une classe sert à cibler plusieurs éléments ; un id identifie un élément.

Comment lire une règle CSS

Un sélecteur indique quels éléments modifier

```
.carte {
  background-color: white;
  padding: 16px;
}
```

.carte

Cible les éléments portant class="carte".

Propriété et valeur

background-color définit le fond.
padding ajoute de l’espace intérieur.

Les accolades regroupent les déclarations de la règle.

Flexbox organise les éléments sur un axe

Une disposition pratique pour une navigation

Un conteneur

display: flex s’applique au parent. Ses enfants directs deviennent des éléments flex.

```
.barre {
  display: flex;
}
```

Dans notre page

La barre contient le nom du site et la navigation.

Par défaut, les éléments se placent sur une ligne.

Les propriétés Flexbox de la barre

Les règles de notre exercice

```
.barre {
  display: flex;
  justify-content: space-between;
  align-items: center;
}
nav { display: flex; gap: 16px; }
```

space-between

Répartit l'espace disponible entre le nom du site et la navigation.

align-items: center

Aligne les éléments sur l'axe transversal.

Dans nav, gap ajoute un espace entre les liens.

CSS Grid organise les cartes en grille

Des colonnes et des lignes pour les contenus

Un conteneur

display: grid transforme les enfants directs en éléments de la grille.

```
.documents {
  display: grid;
  grid-template-columns:
    repeat(3, 1fr);
  gap: 16px;
}
```

Dans notre page

La section documents contient les cartes.

Comprendre repeat(3, 1fr)

La largeur disponible est partagée entre les colonnes

```
grid-template-columns: repeat(3, 1fr);
```

3 Le nombre de colonnes.

1fr Une fraction égale de l’espace disponible.

gap L’espace entre les cartes.

Adapter la page aux petits écrans

Une media query applique des règles selon la largeur

```
@media (max-width: 640px) {
  .barre {
    flex-direction: column;
  }
  .documents {
    grid-template-columns: 1fr;
  }
}
```

640 px ou moins

La règle s’applique lorsque la fenêtre est assez étroite.

Résultat

La barre devient verticale et les cartes passent sur une colonne.

La balise viewport aide le navigateur mobile à utiliser la largeur de l’écran.

Préparation des fichiers

Un éditeur de code et un navigateur suffisent

1 **Créer un dossier exercice-html-css**

2 **Créer les deux fichiers dans ce dossier**

```
exercice-html-css/
  index.html
  style.css
```

Pour cet exercice, nous écrivons le CSS sans Bootstrap.

La structure du fichier HTML

index.html · partie 1 sur 3

```
<!DOCTYPE html>
<html lang="fr">
<head>
  <meta charset="UTF-8">
  <meta name="viewport"
        content="width=device-width, initial-scale=1">
  <title>Mes documents</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>
```

Le lien charge la feuille CSS située dans le même dossier.

L’en-tête et la navigation

index.html · partie 2 sur 3

```
<header class="barre">
  <strong>Mes documents</strong>
  <nav aria-label="Navigation principale">
    <a href="#documents">Documents</a>
    <a href="#ajouter">Ajouter</a>
  </nav>
</header>
```

Les liens rejoindront les sections portant ces identifiants.

Le contenu principal

index.html · partie 3 sur 3

```
  <main>
    <h1>Mes documents</h1>
    <p>Retrouvez vos documents dans cet espace.</p>
    <section id="documents" class="documents">
      <article class="carte"><h2>Contrat</h2><p>PDF</p></article>
      <article class="carte"><h2>Facture</h2><p>PDF</p></article>
      <article class="carte"><h2>Rapport</h2><p>PDF</p></article>
    </section>
    <section id="ajouter">
      <h2>Ajouter un document</h2>
      <p>Le formulaire sera ajouté plus tard.</p>
    </section>
  </main>
</body>
</html>
```

Travail pratique CSS

Dans style.css, mettez en forme la page

01 Styles de base

Fond clair, police lisible et cartes blanches.

02 Flexbox

Aligner le nom du site et les liens de navigation.

03 CSS Grid et affichage mobile

Trois colonnes sur grand écran, une seule sur petit écran.

Enregistrez puis actualisez le navigateur après vos modifications.

Correction 1 Styles de base

style.css · partie 1 sur 4

```
* { box-sizing: border-box; }
body {
  margin: 0;
  font-family: Arial, sans-serif;
  color: #1f2937;
  background-color: #f3f4f6;
}
main {
  max-width: 1000px;
  margin: 0 auto;
  padding: 24px;
}
```

Le contenu principal reste centré et sa largeur est limitée.

Correction 2 — Flexbox

style.css · partie 2 sur 4

```
.barre {
  display: flex;
  justify-content: space-between;
  align-items: center;
  padding: 16px 24px;
  background-color: #dbeafe;
}
nav { display: flex; gap: 16px; }
nav a { color: #1e3a8a; }
```

Flexbox aligne les éléments de l’en-tête sur une ligne.

Correction 3 CSS Grid

style.css · partie 3 sur 4

```
.documents {
  display: grid;
  grid-template-columns: repeat(3, 1fr);
  gap: 16px;
}
.carte {
  background-color: white;
  padding: 16px;
  border-radius: 8px;
}
```

La grille répartit les cartes dans trois colonnes de même largeur.

Correction 4 Petits écrans

style.css · partie 4 sur 4

```
@media (max-width: 640px) {
  .barre {
    flex-direction: column;
    align-items: flex-start;
    gap: 12px;
  }
  .documents {
    grid-template-columns: 1fr;
  }
}
```

À 640 px ou moins, les cartes passent sur une seule colonne.

Vérification et questions

Ouvrez index.html et redimensionnez la fenêtre

Sur grand écran

Les trois cartes sont côte à côte.
La navigation est alignée.

Sur petit écran

Les cartes passent les unes sous les autres,
sans débordement.

À essayer : ajoutez une quatrième carte et observez son placement.

L’interface est prête

Les participants disposent déjà des fichiers du projet

Application Mes documents

Une interface Bootstrap avec navigation, liste et formulaire.

Ouvrir dans le navigateur

```
apps/app-html-css-js/index.html
```

Nous allons comprendre le JavaScript qui anime cette interface.

Bootstrap dans notre interface

Une bibliothèque CSS et JavaScript prête à utiliser

Bootstrap propose des classes pour organiser la page et mettre en forme les composants.

Dans notre application

Navigation responsive, colonnes, cartes, formulaire et boutons.

Les classes Bootstrap sont complétées par notre fichier style.css.

Charger Bootstrap en local

Les fichiers sont déjà fournis dans le projet

```
<link rel="stylesheet"
  href="assets/vendor/bootstrap/bootstrap.min.css">
<link rel="stylesheet"
  href="assets/css/style.css">

<script src="assets/vendor/bootstrap/bootstrap.bundle.min.js" defer>
</script>
```

Le CSS Bootstrap est chargé avant notre feuille personnalisée.

Le fichier bundle active les composants JavaScript comme le menu mobile.

La grille Bootstrap

Une ligne peut être répartie en 12 colonnes

```
<div class="container">
  <div class="row g-4">
    <section class="col-12 col-lg-7">...</section>
    <section class="col-12 col-lg-5">...</section>
  </div>
</div>
```

container

Cadre et centre le contenu.

row et col

Organisent les éléments en colonnes.

De lg à xl : 7 + 5 = 12. En dessous : chaque section occupe toute la largeur.

Les points de rupture

La disposition change selon la largeur de la fenêtre

col-12	La section occupe toute la largeur.
col-lg-7	À partir de lg, la liste occupe 7 colonnes.
col-xl-8	À partir de xl, elle occupe 8 colonnes.

Essayez en redimensionnant la fenêtre du navigateur.

La navigation responsive

Un menu horizontal qui se replie sur petit écran

```
<nav class="navbar navbar-expand-md navbar-dark">
  <button class="navbar-toggler"
    data-bs-toggle="collapse"
    data-bs-target="#navigation">...</button>
  <div class="collapse navbar-collapse"
    id="navigation">...</div>
</nav>
```

navbar-expand-md

Menu déplié à partir de md.

collapse

Le bouton commande le menu.

Les attributs relient le bouton au menu identifié par navigation.

Les cartes et la liste

Bootstrap structure les blocs, notre CSS personnalise les lignes

```
<div class="card app-card">
  <div class="card-body p-4">
    <h2>Liste des documents</h2>
    <div id="document-list">
      <article class="document-item">...</article>
    </div>
  </div>
</div>
```

card et card-body

Composants Bootstrap.

app-card

Classe définie dans style.css.

document-item est également une classe de notre application.

Le formulaire Bootstrap

Les champs utilisent des classes de mise en forme

```
<label class="form-label"
  for="document-title">Titre du document</label>
<input class="form-control"
  id="document-title" required>

<select class="form-select"
  id="document-category" required>...</select>

<button class="btn btn-primary">
  Ajouter le document
</button>
```

label relie le texte au champ. required indique qu’il doit être renseigné.

Les classes utilitaires

Quelques exemples présents dans notre code

d-flex	Dispose les enfants avec Flexbox.
gap-3	Ajoute un espace entre les éléments.
ms-auto	Pousse un élément vers la fin de la ligne.
p-4 et mb-3	Ajoutent des espacements intérieurs ou inférieurs.
w-100	Donne une largeur de 100 %.

Exercice Bootstrap

Observer et modifier l’interface fournie

01 **Repérer les deux colonnes**

Identifier les classes responsive de la liste et du formulaire.

02 **Tester le menu mobile**

Réduire la fenêtre, ouvrir le menu puis choisir un lien.

03 **Ajouter une carte fictive**

Copier une ligne de document dans le HTML et changer son titre.

Corriger ensemble : quelle classe place les sections côte à côte ?

Le rôle de JavaScript

Ajouter des interactions à la page

```
<script src="assets/js/app.js" defer></script>
```

Dans index.html, cette ligne charge le fichier JavaScript.

defer attend la lecture du HTML avant d’exécuter le script.

Le script réagit au formulaire et met à jour la liste.

Variables et constantes

Conserver des références aux éléments HTML

```
const form = document.querySelector('#document-form');
const titleInput = document.querySelector('#document-title');
const fileInput = document.querySelector('#document-file');
```

const Déclare une constante : la référence ne sera pas réaffectée.

Ces constantes désignent des éléments de la page, dont on peut lire ou modifier les propriétés.

Le DOM et les sélecteurs

Retrouver les éléments de la page

```
const documentList = document.querySelector('#document-list');
const count = documentList.querySelectorAll('.document-item').length;
```

querySelector

Récupère le premier élément correspondant au sélecteur.

querySelectorAll

Récupère les éléments correspondants. length donne leur nombre.

cible un identifiant ; . cible une classe.

Fonctions et modèles de chaînes

Mettre le compteur de documents à jour

```
function updateDocumentCount() {
  const count = documentList
    .querySelectorAll('.document-item').length;
  documentCount.textContent =
    `${count} document${count > 1 ? 's' : ''}`;
}
```

Une fonction

Regroupe des instructions réutilisables.

Une chaîne modèle

Les expressions ${...} insèrent des valeurs dans le texte.

La condition ajoute « s » lorsque count est supérieur à 1.

Événements et formulaire

Réagir à l’envoi du formulaire

```
form.addEventListener('submit', (event) => {
  event.preventDefault();
  // Suite du traitement du formulaire
});
```

addEventListener

Écoute un événement et lance une fonction.

preventDefault

Empêche ici le rechargement habituel de la page.

La fonction fléchée reçoit l’événement dans le paramètre event.

Valider les champs

Vérifier le titre et la catégorie avant l'ajout

```
const title = titleInput.value.trim();
titleInput.setCustomValidity(
  title.length < 3 ? 'Saisissez au moins 3 caractères.' : ''
 );
form.classList.add('was-validated');
if (!form.checkValidity()) {
  form.querySelector(':invalid')?.focus();
  return;
}
```

trim

Retire les espaces au début et à la fin.

checkValidity

Contrôle les contraintes du formulaire.

Si un champ est invalide, le traitement s'arrête.

Créer des éléments dans le DOM

Construire une nouvelle ligne de document

```
const article = document.createElement('article');
article.className = 'document-item';

const heading = document.createElement('h3');
heading.textContent = title;
```

createElement

Crée un élément HTML en mémoire.

textContent

Insère le titre comme du texte.

Le titre saisi n’est pas interprété comme du code HTML.

Assembler puis insérer la ligne

Ajouter le document au début de la liste

```
// Dans createDocument
details.append(heading, metadata);
article.append(icon, details, categoryLabel);
return article;

// Dans le gestionnaire submit
documentList.prepend(createDocument(title, category, file));
updateDocumentCount();
```

append ajoute des enfants ; prepend insère la ligne en premier.

Lire le fichier sélectionné

Afficher son nom dans la liste

```
// Dans le gestionnaire submit
const category = categoryInput.value;
const file = fileInput.files[0];

// Dans createDocument(title, category, file)
metadata.textContent = file
  ? `${category} · ${file.name}`
  : `${category} · Ajouté à l'instant`;
```

files[0]

Désigne le premier fichier sélectionné, ou aucune valeur.

file.name

Fournit uniquement le nom affiché dans la page.

Le fichier n'est ni envoyé ni enregistré à cette étape.

Réinitialiser et confirmer

Terminer l’interaction avec le formulaire

```
form.reset();
form.classList.remove('was-validated');
titleInput.setCustomValidity('');
formMessage.textContent =
  'Document ajouté à la liste pour cette démonstration.';
formMessage.classList.remove('d-none');
```

reset() rétablit les valeurs initiales. Retirer d-none affiche le message.

Un rechargement de la page rétablit les trois documents fictifs.

Le menu mobile

JavaScript utilise le composant Collapse de Bootstrap

```
link.addEventListener('click', () => {
  const navigation = document.querySelector('#navigation');
  if (navigation.classList.contains('show')) {
    bootstrap.Collapse.getOrCreateInstance(navigation).hide();
  }
});
```

Après un clic sur un lien, le menu ouvert se referme.

Bootstrap gère l’ouverture et la fermeture du menu responsive.

Exercice JavaScript

Reprendre le code avec les participants

01 Repérer les constantes et les éléments HTML associés.

02 Expliquer ce qui se passe à l’envoi du formulaire.

03 Ajouter un document et observer le compteur.

04 Recharger la page : que deviennent les ajouts ?

À retenir

Les notions utilisées dans notre application

Variables et fonctions

Organiser les données et les traitements.

DOM et événements

Lire la page et réagir aux actions.

Validation et affichage

Contrôler le formulaire et modifier la liste.

L’enregistrement et l’API seront abordés avec Laravel.

Ressources locales et CDN

D’où proviennent les fichiers utilisés par la page ?

Ressource locale

Le fichier est présent dans notre projet.

CDN

Le fichier est fourni par un service externe via le réseau.

```
assets/vendor/bootstrap/bootstrap.min.css
```

Notre copie de Bootstrap permet d’ouvrir l’interface hors ligne.

Un environnement isolé

Les ressources nécessaires doivent être disponibles sur place

Un environnement Air-Gapped est séparé des réseaux extérieurs.

Notre exemple

Les fichiers Bootstrap sont déjà dans le dossier du projet.

Une application locale ne signifie pas forcément que l'ordinateur est isolé.

localStorage

Mémoriser une préférence dans le navigateur

Dans la copie de démonstration, l’utilisateur active « Affichage compact ».

```
const storageKey = 'formation_affichage_compact';
localStorage.setItem(storageKey, 'oui');
localStorage.getItem(storageKey);
```

Ces valeurs restent généralement après rechargement, selon les réglages du navigateur.

localStorage dans notre application

Comprendre le code du mode compact

```
function appliquerAffichage(compact) {
  documentList.classList.toggle('compact', compact);
  compactButton.checked = compact;
}

appliquerAffichage(
  localStorage.getItem(storageKey) === 'oui'
);
```

La classe compact modifie l’espacement des lignes avec le CSS.

Quelles données conserver ?

localStorage reste accessible aux scripts de la page

Adapté à notre exemple

Une préférence d’affichage non sensible.

À ne pas y placer

Mots de passe et identifiants de session sensibles.

Les documents de notre liste restent temporaires et ne sont pas enregistrés.

Démonstration localStorage

Utiliser la copie app-html-css-js-securite

01 Activer le mode compact

Observer la différence dans la liste.

02 Recharger la page

La préférence reste si le stockage est disponible.

03 Ouvrir les outils de développement

Application > Stockage local > formation_affichage_compact.

Le résultat en file:// peut varier selon le navigateur.

Exercice de synthèse

Mettre en pratique les notions du jour

01 HTML et CSS

Repérer une section et une classe personnalisée.

02 Bootstrap

Retrouver les colonnes et tester le menu mobile.

03 JavaScript

Ajouter un document et expliquer la mise à jour de la liste.

Bonus : activer le mode compact dans la copie de sécurité.

Correction de la synthèse

Quelques éléments à retrouver dans le projet

Structure	section, document-item et style.css.
Responsive	row, col-12, col-lg-7 et navbar-expand-md.
Interactions	submit, createDocument, prepend et updateDocumentCount.

L’ajout reste temporaire : recharger la page rétablit la liste initiale.

Questions et bilan

À la fin de cette journée

Je sais lire la structure d’une page et reconnaître une règle CSS.

Je sais expliquer comment Bootstrap adapte l’interface.

Je sais suivre une interaction JavaScript du formulaire à la liste.

Prochaine étape : construire l’API Laravel.